Language support for empty objects

ISO/IEC JTC1 SC22 WG21
P0840R1
Richard Smith
richard@metafoo.co.uk
2017-11-27

For motivation, see P0840R0.

Proposal

We propose the addition of an attribute, `[[no_unique_address]]` to indicates that a unique address is not required for an non-static data member of a class. A non-static data member with this attribute may share its address (or its tail padding) with another object, if it could when used as a base class.

FAQ

▶ Q: Does this allow multiple subobjects *of the same type* to have the same address?
▶ Q: Can a standard library switch from EBO to this attribute without an ABI break?
▶ Q: Does this allow reuse of tail padding? (Eg, three bytes at the end of `struct A { int n; char c; };`)
▶ Q: When is a type using this attribute considered "empty"? (As visible via `std::is_empty`)
▶ Q: Does the attribute affect whether a type is standard-layout?
▶ Q: Does the attribute affect the "common initial sequence" rule?
▶ Q: Suppose I have members `a`, `b`, `c` (in that order, with the same access). Today we guarantee that `&a < &b < &c`. What happens if `b` has the attribute?
▶ Q: Applying a `[[no_unique_address]]` attribute to a function, constant variable, etc, could be used to permit ICF ("Identical Code Folding" / "Identical Constant Folding" / "Identical COMDAT Folding"). Is this proposed?

Wording

Add a new paragraph before [intro.object] (4.5) paragraph 7:

> A *potentially-overlapping subobject* is either:
>
> - a base class subobject, or
> - a bit-field ([class.bit]), or
> - a non-static data member declared with the `no_unique_address` attribute ([dcl.attr.nouniqueaddr]).

Change in [intro.object] (4.5) paragraph 7:

> **An object has nonzero size if**
>
> - **it is not a potentially-overlapping subobject, or**
> - **it is a bit-field with nonzero length, or**
> - **has subobjects of nonzero size, or**
> - **it is of a class type with virtual member functions or virtual base classes.**
>
> **Otherwise, if the object is a base class subobject of a standard-layout class type, it has zero size. Otherwise, the circumstances under which the object has zero size are implementation-defined.** Unless it is a bit-field (12.2.4), ~~a most derived~~ **an** object ~~shall have a~~ **with** nonzero size ~~and~~ shall occupy one or more bytes of storage**, including every byte that is occupied in full or in part by any of its subobjects**. ~~Base class subobjects may have zero size~~. An object of trivially

copyable or standard-layout type (6.9) shall occupy contiguous bytes of storage. **A bit-field of length *N* shall occupy *N* bits of storage.**

Change in [intro.object] (4.5) paragraph 8:

Unless an object is a bit-field or a ~~base class~~ subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects ~~a and b~~ with overlapping lifetimes that are not bit-fields may have the same address if one is nested within the other, or if at least one is a ~~base class~~ subobject of zero size and they are of different types; otherwise, they have distinct addresses **and shall occupy disjoint bytes of storage**. [Footnote] [Example] **The bits occupied by a bit-field shall be disjoint from the bits or bytes occupied by any other object with overlapping lifetime that the bit-field is not nested within.**

Change in [expr.sizeof] (8.3.3) paragraph 1:

The `sizeof` operator yields the number of bytes **occupied by a non-potentially-overlapping object of the type** ~~in the object representation~~ of its operand. […]

Change in [expr.sizeof] (8.3.3) paragraph 2:

When applied to a reference or a reference type, the result is the size of the referenced type. When applied to a class, the result is the number of bytes in an object of that class including any padding required for placing objects of that type in an array. ~~The size of a most derived class shall be greater than zero (4.5).~~ The result of applying `sizeof` to a ~~base class~~ **potentially-overlapping** subobject is the size of the ~~base class~~ type**, not the size of the subobject**. [Footnote: The actual size of a ~~base class~~ **potentially-overlapping** subobject may be less than the result of applying `sizeof` to the subobject, due to virtual base classes and less strict padding requirements on ~~base class~~ **potentially-overlapping** subobjects.] When applied to an array, the result is the total number of bytes in the array. This implies that the size of an array of n elements is n times the size of an element.

Change in [expr.rel] (8.9) paragraph 3:

Comparing unequal pointers to objects is defined as follows:

- […]
- If two pointers point to different non-static data members of the same object, or to subobjects of such members, recursively, the pointer to the later declared member compares greater provided the two members have the same access control (Clause 14)**, neither member is a subobject of zero size,** and ~~provided~~ their class is not a union.
- Otherwise, neither pointer compares greater than the other.

Add a new subclause [dcl.attr.nouniqueaddr] after [dcl.attr.noreturn]:

10.6.9 No unique address attribute [dcl.attr.nouniqueaddr]

The *attribute-token* `no_unique_address` specifies that a non-static data member need not have an address distinct from all other non-static data members of its class. It shall appear at most once in

each *attribute-list* and no *attribute-argument-clause* shall be present. The attribute may appertain to a non-static data member other than a bit-field.

[*Note*: The non-static data member can share the address of another non-static data member of that of a base class, and any padding that would normally be inserted at the end of the object can be reused as storage for other members. — *end note*] [*Example*:

```
template<typename Key, typename Value,
         typename Hash, typename Pred, typename Allocator>
class hash_map {
  [[no_unique_address]] Hash hasher;
  [[no_unique_address]] Pred pred;
  [[no_unique_address]] Allocator alloc;
  Bucket *buckets;
  // ...
public:
  // ...
};
```

Here, `hasher`, `pred`, and `alloc` could have the same address as `buckets` if their respective types are all empty. — *end example*]

Change in [class] (12) paragraph 4 and split into two paragraphs:

[Note: Complete objects ~~and member subobjects~~ of class type shall have nonzero size. ~~[~~ ~~Footnote:~~ Base class subobjects **and members declared with the `no_unique_address` attribute ([dcl.attr.nouniqueaddr])** are not so constrained.]

[Note: ...]

Change in [class.mem] (12.2) paragraph 21:

The *common initial sequence* of two standard-layout struct (Clause 12) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that corresponding entities have layout-compatible types**, either neither entity is declared with the `no_unique_address` attribute ([dcl.attr.nouniqueaddr]]) or both are, and** and either neither entity is a bit-field or both are bit-fields with the same width. [Example]

Change in [meta.unary.prop] (23.15.4.3) Table 42 ("Type property predicates"):

Template	Condition	Predicate
	…	
`template <class T> struct is_empty;`	`T` is a class type, but not a union type, with no non-static data members other than ~~bit-fields of length 0~~ **subobjects of zero size,** no virtual member functions, no virtual base classes, and no base class `B` for which `is_empty_v<B>` is `false`.	If `T` is a non-union class type, `T` shall be a complete type.
	…	